

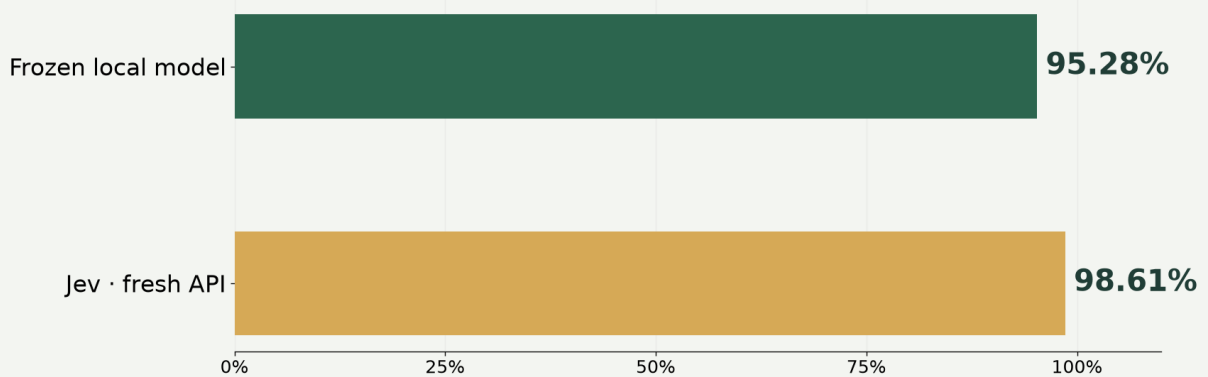
Can Astra Build Its Own Jev?

FILMING GUIDE / 21 SEPTEMBER 2026

The photo-aware local agency, the frozen text challenger, and the measured results.

The stronger model still lost.

360 fresh scenarios · 1,440 policy decisions per engine · zero request failures



Local minus Jev: -3.33 percentage points | 95% interval: -4.38 to -2.29

Synthetic, agent-reviewed references. Two writer-assisted adjudications; no human validation.

Source: round1-deeper-fp16 paired report · frozen before generation · all prespecified cases retained

V2 was not promoted.

The fresh synthetic travel result was 95.28% local versus 98.61% Jev. The app still serves historical V1. Reference repairs and two writer-assisted adjudications are disclosed; no human validation.

2,265 spoken words. 14.6 minutes at 155 WPM, with a planned 18.7-minute runtime including visual holds. Script-only narration source is supplied; no TTS requested.

0:00 - 0:55 / See how a photo changes the answer

PICTURE

Scene `#demo` at localhost:8770. Start with the live persona demo already visible. Deliver the Jev/local/images claim over the app. Run Maya with the stairs photo by 0:10; remove it and run again. Point at review by 0:25. Briefly show the full agency grid. If either request stalls, preserve real elapsed time and cut the wait transparently.

SAY

I took the idea behind Jev and built a version for this travel job that runs entirely on my computer. And I even added the ability to understand images, which is something Jev's current model doesn't support. Now imagine you run a travel agency. You've got twelve customers, all with different budgets and wish lists. One needs a pool. Another arrives after midnight. Maya wants a cash refund if she cancels, and an entrance without stairs. Your job is to match each person with a holiday that fits, checking the fine print and the photos. Watch what happens when we check this one.

ON-SCREEN COPY

See how a photo changes the answer.

EDITING NOTE

Let the result finish before speaking its meaning. P pauses diagrams; R resets current scene. Give the final visual time to read. Setup commands and photo results must be real; extended installation waits can be cut with the elapsed time disclosed.

SOURCE / TRUTH CARD

Fresh /api/decide receipts; active V1 plus pretrained vision. Fictional travellers and photos. TypeSafe model documentation checked 21 September 2026: current Jev text-only input; our image support is a separate pretrained integration.

0:55 - 1:40 / Follow the build from start to finish

PICTURE

Scene `#why` at localhost:8770. Trace the six steps. Open the full guide briefly, then return to the first step.

SAY

The written terms haven't changed. The picture gave our system something else to consider. Now I'll show you how to build this kind of system yourself, using AI to help with the code. We'll choose an existing model, download it, give Codex a detailed brief, train it on examples, and test it. Then we'll connect the pictures. I'm giving you the code, model downloads and the full guide. You don't need to know machine learning to follow the explanation. You do need to check that the examples teach the job you actually want.

ON-SCREEN COPY

Follow the build from start to finish.

EDITING NOTE

Let the result finish before speaking its meaning. P pauses diagrams; R resets current scene. Give the final visual time to read. Setup commands and photo results must be real; extended installation waits can be cut with the elapsed time disclosed.

SOURCE / TRUTH CARD

docs/STEP-BY-STEP.md; exact tutorial commands and retained app.

1:40 - 2:25 / Start with an existing classifier

PICTURE

Scene `#choose-model` at localhost:8770. Open the actual model card. Point to owner, task, weights and pinned revision; use the card as a navigation surface, not a fabricated screenshot.

SAY

First, find a model that can already do the kind of job you need. Hugging Face hosts model weights and instructions. This is the one behind our travel demo: Moritz Laurer's ModernBERT zero-shot classifier. Someone has already trained it to compare text with possible answers. We aren't starting from a blank model. On the model page, check what it does, whether you can download it, its terms, and its results. Save the exact version. Codex can help you read the page, but a popular model still needs testing on your job.

ON-SCREEN COPY

Start with an existing classifier.

EDITING NOTE

Let the result finish before speaking its meaning. P pauses diagrams; R resets current scene. Give the final visual time to read. Setup commands and photo results must be real; extended installation waits can be cut with the elapsed time disclosed.

SOURCE / TRUTH CARD

Pinned Hugging Face author model card; travel_lab/nli.py NLI_ID and NLI_REV.

2:25 - 3:10 / Measure it before you train it

PICTURE

Scene `#baseline` at localhost:8770. Point at the before bar first, then after. Keep development and separate final-test caveat visible. Open the JSON receipt.

SAY

Before changing it, find out what it can already do. On our travel development examples, the starting classifier got about seventy-five percent right. After travel training, it got about ninety-four percent on those same examples. So training changed something useful on that set. But we used those development results to choose the model. They aren't an independent final score. The earlier model only got sixty-six point five percent on its separate final test. Keep those receipts separate. You'll measure your own baseline before training, then use untouched examples to find out whether the improvement holds.

ON-SCREEN COPY

Measure it before you train it.

EDITING NOTE

Let the result finish before speaking its meaning. P pauses diagrams; R resets current scene. Give the final visual time to read. Setup commands and photo results must be real; extended installation waits can be cut with the elapsed time disclosed.

SOURCE / TRUTH CARD

models/selection.json baseline_dev and dev; runs/nli-baseline-dev.json; historical final summary. 800 development judgments.

3:10 - 3:40 / Check your computer first

PICTURE

Scene `#setup` at localhost:8770. Show tested M5 Max/128 GB host and separate text/image requirements. Open Start here prerequisites briefly.

SAY

Check your computer before downloading. I used an M5 Max with a hundred and twenty-eight gigabytes of memory. That's the tested machine, not a minimum. The supplied image setup needs Apple silicon and about sixteen gigabytes of model downloads. That's disk space, not a RAM requirement. Smaller machines and Windows or Linux image setups aren't validated here. Text and pictures have separate setup steps.

ON-SCREEN COPY

Check your computer first.

EDITING NOTE

Let the result finish before speaking its meaning. P pauses diagrams; R resets current scene. Give the final visual time to read. Setup commands and photo results must be real; extended installation waits can be cut with the elapsed time disclosed.

SOURCE / TRUTH CARD

Inspected host, macOS 26.5.2; pinned vision launcher. No unsupported hardware promises.

3:40 - 4:25 / Download the model to your computer

PICTURE

Scene `#download` at localhost:8770. Show the source repository in Codex, then run the download command in its terminal. Show actual printed revision and cache path. If cached, say so; do not stage a fresh download.

SAY

Open the project folder in Codex and follow the setup page to install its tools. Then run this command from the repository root. It downloads the exact model version, its tokenizer and its configuration. Weights are the saved model settings. The tokenizer turns text into the pieces the model reads. You should get a model name, a version and a local folder. If you've downloaded it before, this can reuse the cache. Internet is needed for the first download. Nothing has been fine-tuned yet. We've put the starting model on the computer.

ON-SCREEN COPY

Download the model to your computer.

EDITING NOTE

Let the result finish before speaking its meaning. P pauses diagrams; R resets current scene. Give the final visual time to read. Setup commands and photo results must be real; extended installation waits can be cut with the elapsed time disclosed.

SOURCE / TRUTH CARD

tools/tutorial.py download; live pinned snapshot_download receipt; docs/STEP-BY-STEP.md.

4:25 - 5:05 / Tell Codex exactly what you want

PICTURE

Scene `#build` at localhost:8770. Point at each requirement before showing the full prompt. Keep Codex visibly open on the repository when filming.

SAY

Now ask Codex to help turn this into a specialist. Saying fine-tune a model for travel leaves too much to guess. Tell it the exact job, what text it will receive, which answers it can choose, and what should happen when the evidence is missing. Give it your computer constraints and ask for a small test before a large run. Codex can write and run code, but it needs your definition of a correct answer. Let me show you the travel version, then the full prompt you can copy.

ON-SCREEN COPY

Tell Codex exactly what you want.

EDITING NOTE

Let the result finish before speaking its meaning. P pauses diagrams; R resets current scene. Give the final visual time to read. Setup commands and photo results must be real; extended installation waits can be cut with the elapsed time disclosed.

SOURCE / TRUTH CARD

prompts/TRAIN-MY-SPECIALIST.md; supplied travel hypotheses and task data.

5:05 - 5:50 / Write the rules for your travel job

PICTURE

Scene `#travel-brief` at localhost:8770. Underline the label contract and four definitions one by one while explaining. These are real travel requirements from the current model.

SAY

For our travel job, we ask four separate questions. Is there a full cash refund before the deadline? Can the guest arrive after midnight without arranging it first? Is pool access included? Is a guided hike included? Each question gets one of the same three answers. Notice how specific the definitions are. Hotel credit doesn't satisfy the cash-refund rule. A pool in a picture doesn't establish free access. And a missing clause doesn't become a yes. Your own brief needs these kinds of boundaries, because those are the answers your training examples will teach.

ON-SCREEN COPY

Write the rules for your travel job.

EDITING NOTE

Let the result finish before speaking its meaning. P pauses diagrams; R resets current scene. Give the final visual time to read. Setup commands and photo results must be real; extended installation waits can be cut with the elapsed time disclosed.

SOURCE / TRUTH CARD

travel_lab/nli.py HYPOTHESES; traveller requirements; text/image separation.

5:50 - 6:45 / Give Codex the complete brief

PICTURE

Scene `#prompt` at localhost:8770. Click Job, Data, Training and Final test tabs. Open the complete prompt page and show that the download has all eight sections. Underlines identify editable task fields.

SAY

Here's the complete prompt. It's long because it's doing more than asking for a model. The first section defines your job. Change the bracketed parts. The data section tells Codex to show you examples and keep related documents together. The training section asks for a tiny working run before the full one. The final-test section says to freeze the model and keep the mistakes. Keep those requirements when you change domains. Download the whole prompt, paste it into Codex with the repository open, and have it walk through each checkpoint with you. This is a reusable specification; my original project took follow-ups and fixes.

ON-SCREEN COPY

Give Codex the complete brief.

EDITING NOTE

Let the result finish before speaking its meaning. P pauses diagrams; R resets current scene. Give the final visual time to read. Setup commands and photo results must be real; extended installation waits can be cut with the elapsed time disclosed.

SOURCE / TRUTH CARD

Exact downloadable eight-part prompt; prepared specification, not historical chat.

6:45 - 7:15 / Give it three choices

PICTURE

Scene `#labels` at localhost:8770. Click Cash back, then Hotel credit. Point to the mapping between the teaching labels and actual output labels.

SAY

This policy offers hotel credit. Maya wants a cash refund, so it fails her requirement. A clear cash refund passes. If the policy doesn't say, we can't tell. Those are our three labels. A classifier picks a label. That gives our app a short answer it can use. It can still pick the wrong one.

ON-SCREEN COPY

Give it three choices.

EDITING NOTE

Let the result finish before speaking its meaning. P pauses diagrams; R resets current scene. Give the final visual time to read. Setup commands and photo results must be real; extended installation waits can be cut with the elapsed time disclosed.

SOURCE / TRUTH CARD

Teaching exercise; actual NLI labels in data/train.jsonl. Fixed format is not a deterministic-output claim.

7:15 - 7:50 / Pair each example with an answer

PICTURE

Scene `#training` at localhost:8770. Open actual train-00000 excerpt. Download the full record. Point at input, question, answer; show the separate held-out file only as data provenance.

SAY

We started with ModernBERT, a model that already understands language patterns. To make it better at this job, we gave it travel examples. Each has three parts. What it reads, what we ask, and the checked answer. This actual record says to contact the booking office. So we can't establish the refund rule. Our examples are made up. For your job, check the labels carefully. Wrong answers teach the wrong lesson.

ON-SCREEN COPY

Pair each example with an answer.

EDITING NOTE

Let the result finish before speaking its meaning. P pauses diagrams; R resets current scene. Give the final visual time to read. Setup commands and photo results must be real; extended installation waits can be cut with the elapsed time disclosed.

SOURCE / TRUTH CARD

assets/training-example.json exact data/train.jsonl record; synthetic, no human labels.

7:50 - 8:30 / Keep three separate sets of examples

PICTURE

Scene `#data-split` at localhost:8770. Show three real data files. Point at scenario ID and template family in source records. Do not inspect final answer errors until the model is frozen.

SAY

Before training, split the examples into three groups. Training examples change the model. Development examples help you choose settings and which checkpoint to keep. The final test stays out of those decisions. If two examples came from the same conversation or nearly identical document template, keep them together. Otherwise you can accidentally test something the model has practically seen already. The preparation tool checks exact overlaps and the file format. It cannot spot every near-copy or decide whether your label makes sense. That's where your knowledge of the job matters.

ON-SCREEN COPY

Keep three separate sets of examples.

EDITING NOTE

Let the result finish before speaking its meaning. P pauses diagrams; R resets current scene. Give the final visual time to read. Setup commands and photo results must be real; extended installation waits can be cut with the elapsed time disclosed.

SOURCE / TRUTH CARD

Tutorial prepare validation; travel data provenance and split design.

8:30 - 9:15 / Train in a separate working folder

PICTURE

Scene `#workspace` at localhost:8770. Run prepare in the repository root; show created data folder and manifest. Demonstrate that a second prepare refuses the existing folder.

SAY

Create a separate workshop folder. This command checks the input format, copies the data and records hashes, which are fingerprints for the files. Your new model will be saved here. The supplied demo stays where it is. The tool refuses to replace an existing workshop, so use a new name for each experiment. For your own task, point the preparation command at your own data folder. Before doing a full run, use the guide's eight-example smoke check in another folder. That proves the commands work; it doesn't tell you the model is good.

ON-SCREEN COPY

Train in a separate working folder.

EDITING NOTE

Let the result finish before speaking its meaning. P pauses diagrams; R resets current scene. Give the final visual time to read. Setup commands and photo results must be real; extended installation waits can be cut with the elapsed time disclosed.

SOURCE / TRUTH CARD

tools/tutorial.py prepare; exclusive directory and data hash checks.

9:15 - 9:45 / Train it using checked examples

PICTURE

Scene `#practice` at localhost:8770. Play the two-lane HyperFrame. Hold at the wrong guess, checked label and saved-settings update. Point to unchanged settings in the normal-check lane.

SAY

The model guesses. Training compares that guess with the label and adjusts its saved settings. Repeat with more examples. That's fine-tuning. A prompt gives instructions for a request. Training changes what's saved. Clicking Check in our app uses that saved model; it doesn't train it again. And I've included the travel weights, which are those saved settings, so you can start by running the example.

ON-SCREEN COPY

Train it using checked examples.

EDITING NOTE

Let the result finish before speaking its meaning. P pauses diagrams; R resets current scene. Give the final visual time to read. Setup commands and photo results must be real; extended installation waits can be cut with the elapsed time disclosed.

SOURCE / TRUTH CARD

Training code and saved checkpoints. Diagram is conceptual, not a measured individual training step.

9:45 - 10:35 / Run the baseline, then start training

PICTURE

Scene `#train-run` at localhost:8770. Show real baseline receipt, actual epoch log and saved checkpoint files from the verified tutorial run. Clearly label eight-decision smoke check; do not present it as the historical full training run.

SAY

First run the unchanged model on development examples. Save that baseline. Then run training. An epoch means one pass through the practice examples. A batch is how many examples it handles together. The learning rate controls how big each adjustment is. Our starting recipe updates the last two layers and the classification head. Codex can explain or adjust those settings for your machine. Watch the actual loss and development scores, then inspect the checkpoint folder. A falling training loss alone doesn't prove success. This recipe saves the checkpoint with the best development score, ready for a separate final test.

ON-SCREEN COPY

Run the baseline, then start training.

EDITING NOTE

Let the result finish before speaking its meaning. P pauses diagrams; R resets current scene. Give the final visual time to read. Setup commands and photo results must be real; extended installation waits can be cut with the elapsed time disclosed.

SOURCE / TRUTH CARD

tools/tutorial.py actual baseline, training, checkpoint and freeze outputs. New tutorial recipe differs from historical accumulation recipe.

10:35 - 11:05 / Test it on examples it hasn't seen

PICTURE

Scene `#test` at localhost:8770. Show practice and final-test objects. Open the held-out example and explain its split, then return to the diagram.

SAY

Keep some examples out of training, like an exam the model hasn't practised. Keep related documents together when splitting them. Our later test used made-up cases with agent-reviewed answers, including repairs and disputed answers. There was no human validation. We froze the model before scoring it. Once you inspect those mistakes, you need fresh cases for your next final test.

ON-SCREEN COPY

Test it on examples it hasn't seen.

EDITING NOTE

Let the result finish before speaking its meaning. P pauses diagrams; R resets current scene. Give the final visual time to read. Setup commands and photo results must be real; extended installation waits can be cut with the elapsed time disclosed.

SOURCE / TRUTH CARD

Frozen V2 protocol; 52 document repairs; two writer-assisted adjudications; hashes. V1 held-out download is labeled separately.

11:05 - 11:50 / Compare both models on the same test

PICTURE

Scene `#test-run` at localhost:8770. Execute the small verification test if needed; open its saved report. Show rows for both models on the same IDs. Then use historical full-study failure in next scene.

SAY

Once you've chosen the checkpoint, freeze it and open the final test. This command runs the original and trained models on the same inputs and records both answers. Accuracy tells you how often each matched the reference. Macro-F1 also helps when some labels appear much more often than others. Open the individual mistakes, not only the average. If training made things worse, that's the result. Keep it. You can use what you learned for the next experiment, but those inspected cases can't count as a fresh final exam again.

ON-SCREEN COPY

Compare both models on the same test.

EDITING NOTE

Let the result finish before speaking its meaning. P pauses diagrams; R resets current scene. Give the final visual time to read. Setup commands and photo results must be real; extended installation waits can be cut with the elapsed time disclosed.

SOURCE / TRUTH CARD

tools/tutorial.py test; paired rows, frozen model and data hash enforcement.

11:50 - 12:20 / Look at an answer it got wrong

PICTURE

Scene `#failure` at localhost:8770. Show case ...0036 arrival excerpt and both saved outputs. Open full JSON to show the input is longer than the excerpt.

SAY

Here's a real mistake from that test. Arrival at one thirty in the morning is allowed without calling ahead. The reference says yes. Jev says yes. My later model says it can't tell. That sends a clear case to unnecessary review. Both received the same full document. We're looking at an excerpt. This is why I want you to inspect the mistakes too.

ON-SCREEN COPY

Look at an answer it got wrong.

EDITING NOTE

Let the result finish before speaking its meaning. P pauses diagrams; R resets current scene. Give the final visual time to read. Setup commands and photo results must be real; extended installation waits can be cut with the elapsed time disclosed.

SOURCE / TRUTH CARD

assets/benchmark-case.json; raw frozen local and Jev rows for round1-deeper-fp16-0036, arrival.

12:20 - 12:50 / Compare its answers with Jev's

PICTURE

Scene `#results` at localhost:8770. Show common-axis bars; point to text-only scope and distinguish demo V1 from later V2. Do not animate a V1 / V2 improvement arrow.

SAY

That later model matched our references about ninety-five percent of the time. Jev was closer to ninety-nine. I didn't beat Jev. It failed our promotion rule, so the demo still uses the earlier model. That one scored sixty-six point five on a different test. These aren't comparable improvement scores. And this chart tests text, not picture understanding. We built the workflow. Its reliability still needs work.

ON-SCREEN COPY

Compare its answers with Jev's.

EDITING NOTE

Let the result finish before speaking its meaning. P pauses diagrams; R resets current scene. Give the final visual time to read. Setup commands and photo results must be real; extended installation waits can be cut with the elapsed time disclosed.

SOURCE / TRUTH CARD

FROZEN-V2-RESULTS.md; qualification-decision.json; V1 summary. No model or final-test changes.

12:50 - 13:35 / Try your saved model on new text

PICTURE

Scene `#predict` at localhost:8770. Download the actual input, run predict, read its returned sentence. Change cash refund to hotel credit and explain the expected change without guaranteeing it.

SAY

Now load the saved model and give it a new piece of text. This file has a cancellation policy and three possible answers. Run the prediction command, then read the sentence it selected. Change the policy to hotel credit and try again. You're now using the saved specialist, without asking Codex to answer the holiday question. Codex helped build it. To connect your new checkpoint to an app, give it a separate loading path and test the output mapping. Don't overwrite the working demo's model or assume its old thresholds fit the new one.

ON-SCREEN COPY

Try your saved model on new text.

EDITING NOTE

Let the result finish before speaking its meaning. P pauses diagrams; R resets current scene. Give the final visual time to read. Setup commands and photo results must be real; extended installation waits can be cut with the elapsed time disclosed.

SOURCE / TRUTH CARD

tools/tutorial.py predict; downloadable JSON; actual smoke prediction receipt.

13:35 - 14:05 / Connect the models with code

PICTURE

Scene `#map` at localhost:8770. Point to text reader, pretrained photo reader and app rules in order; keep Maya visible when returning to the agency.

SAY

There are three pieces. A model reads the terms. Another looks at the pictures. Code checks those answers against Maya's wishes and budget. I trained the text reader for travel and connected an already trained image reader. Multimodal means we're using words and pictures. Price comparisons stay in ordinary code. You don't need AI to tell you whether twelve hundred is more than a thousand.

ON-SCREEN COPY

Connect the models with code.

EDITING NOTE

Let the result finish before speaking its meaning. P pauses diagrams; R resets current scene. Give the final visual time to read. Setup commands and photo results must be real; extended installation waits can be cut with the elapsed time disclosed.

SOURCE / TRUTH CARD

travel_lab/vision.py; catalogue.py; active selection.json.

14:05 - 14:40 / Add a model that reads images

PICTURE

Scene `#vision` at localhost:8770. Play photo / observation / app. Open photo-lab.html, choose a local image, run it and inspect actual answers and image hash. Keep the service running before filming.

SAY

Now connect the model that reads photos. This pretrained reader looks at the actual pixels and answers questions about visible features. I connected it through OpenJev. Pick a PNG or JPEG in the photo lab and run it on this computer. We send the picture, without its filename or an expected answer. This model reads pictures. The holiday illustrations were created separately.

ON-SCREEN COPY

Add a model that reads images.

EDITING NOTE

Let the result finish before speaking its meaning. P pauses diagrams; R resets current scene. Give the final visual time to read. Setup commands and photo results must be real; extended installation waits can be cut with the elapsed time disclosed.

SOURCE / TRUTH CARD

Fresh /api/observe receipt; fixed loopback endpoint; pretrained DiffusionGemma. Generated fictional catalogue photos.

14:40 - 15:10 / Check what the photo can tell you

PICTURE

Scene `#boundary` at localhost:8770. Show pond, ask the three questions and reveal the conclusion. Refer back to the missing stairs photo.

SAY

There's water here, but that doesn't establish a swimming pool. Even a pool photo can't tell us whether access costs extra. Pictures show visible features. Terms tell us what's promised. Our forty offers share nine fictional images, so this isn't a test of forty real hotels. And removing Maya's stairs photo never proves the entrance is accessible.

ON-SCREEN COPY

Check what the photo can tell you.

EDITING NOTE

Let the result finish before speaking its meaning. P pauses diagrams; R resets current scene. Give the final visual time to read. Setup commands and photo results must be real; extended installation waits can be cut with the elapsed time disclosed.

SOURCE / TRUTH CARD

Generated-image ledger; location-photos.json; image evidence limitations.

15:10 - 15:40 / Check each person's requirements

PICTURE

Scene `#rules` at localhost:8770. Play budget/terms/photo gates. In the actual agency open a traveller drawer and show the clause/photo that caused decline or review.

SAY

The app combines the checks. Budget passes. Terms pass. Stairs fail, so Maya gets a decline. Missing evidence goes to review unless another requirement already fails. Open her card and inspect the reason. A green card means these checks passed according to our models. A person still confirms the booking details. The app doesn't book anything.

ON-SCREEN COPY

Check each person's requirements.

EDITING NOTE

Let the result finish before speaking its meaning. P pauses diagrams; R resets current scene. Give the final visual time to read. Setup commands and photo results must be real; extended installation waits can be cut with the elapsed time disclosed.

SOURCE / TRUTH CARD

catalogue.verdict; apply_visual_requirements; live reason drawer; companion local API example.

15:40 - 16:20 / Run it before changing it

PICTURE

Scene `#first-run` at localhost:8770. Open Start here. Show source and restored files, run `first_run.py`, start agency in a spare terminal if needed, inspect service status and run a fresh holiday. No fake installation animation.

SAY

Open Start here. Get the source and matching model archive, then install the listed tools. Run the first-run check to see what's missing. Start the text service, open the app, and add the photo service when ready. The commands are copyable. Check a holiday yourself. Leave the services running; after sleep, check them again. If they're unavailable, the page tells you instead of showing an old answer.

ON-SCREEN COPY

Run it before changing it.

EDITING NOTE

Let the result finish before speaking its meaning. P pauses diagrams; R resets current scene. Give the final visual time to read. Setup commands and photo results must be real; extended installation waits can be cut with the elapsed time disclosed.

SOURCE / TRUTH CARD

`tools/first_run.py`, `tools/restore_companion.py`, `docs/SETUP.md` and live `/api/status`. Public access separately gated.

16:20 - 16:50 / Work out what it costs to run

PICTURE

Scene `#cost` at localhost:8770. Show build/setup and local-running cost columns, then separately labeled historical timings. Point to fresh demo time rather than implying benchmark-equivalent workloads.

SAY

These local checks don't create a model-provider API bill. You still have hardware, electricity, setup and maintenance. Building and training can cost money too. I haven't measured the whole bill or proved a fixed saving. These timings cover different jobs, so compare the same workload on your computer before deciding whether this approach pays off.

ON-SCREEN COPY

Work out what it costs to run.

EDITING NOTE

Let the result finish before speaking its meaning. P pauses diagrams; R resets current scene. Give the final visual time to read. Setup commands and photo results must be real; extended installation waits can be cut with the elapsed time disclosed.

SOURCE / TRUTH CARD

Historical V1 HTTP timing receipts and photo integration scan; no measured total-cost ratio.

16:50 - 17:25 / Change one example yourself

PICTURE

Scene `#repurpose` at localhost:8770. Edit support request, choose its human label, add it and download JSONL. Run `prepare_task.py` on the saved sample. Show held-out separation in the adaptation guide.

SAY

Say your job is sorting support messages. Here's someone charged twice. Choose Billing, add the example, and download the data file. You've made a checked example. You haven't trained a support model yet. The guide shows how to prepare the data, change the task and keep a separate test. Start with cases you understand, including confusing ones. Changing the labels alone won't teach the travel model a new job.

ON-SCREEN COPY

Change one example yourself.

EDITING NOTE

Let the result finish before speaking its meaning. P pauses diagrams; R resets current scene. Give the final visual time to read. Setup commands and photo results must be real; extended installation waits can be cut with the elapsed time disclosed.

SOURCE / TRUTH CARD

Real JSONL export; `tools/prepare_task.py`; `docs/ADAPT-YOUR-OWN.md`. No completed support-model claim.

17:25 - 18:05 / Keep the files that let you run it again

PICTURE

Scene `#handover` at localhost:8770. Show the actual workshop file tree and the README/prompt folder. Explain which files a different domain changes.

SAY

When Codex finishes, ask it to leave the whole project understandable. Keep your task brief, checked data, model version, training settings, saved checkpoint, test predictions and a README with the commands. Keep the failures too. For a new business job, change the questions, candidate descriptions, label rules and examples together, then repeat the baseline and test. The travel API has assumptions that your new app needs to replace. You're reusing a process and working code, not renaming a travel model and hoping it knows your business.

ON-SCREEN COPY

Keep the files that let you run it again.

EDITING NOTE

Let the result finish before speaking its meaning. P pauses diagrams; R resets current scene. Give the final visual time to read. Setup commands and photo results must be real; extended installation waits can be cut with the elapsed time disclosed.

SOURCE / TRUTH CARD

docs/STEP-BY-STEP.md; eight-part prompt completion contract; tutorial output files.

18:05 - 18:40 / Start with a working example

PICTURE

Scene `#take-it` at localhost:8770. Open Start here and the question index. Show current access/permission status accurately; use only the delivered link once verified. Keep original creator/model credits visible.

SAY

Start with the full guide. Download the starting model, copy the complete Codex prompt, run the small check, then train and test your own workshop. The repository includes the code, examples, setup instructions, model downloads and the results we kept, including the losses. Follow the stated reuse terms and keep the upstream credits. You can use AI to help you build the software. Your job is to choose a useful question and check whether the answers make sense.

ON-SCREEN COPY

Start with a working example.

EDITING NOTE

Let the result finish before speaking its meaning. P pauses diagrams; R resets current scene. Give the final visual time to read. Setup commands and photo results must be real; extended installation waits can be cut with the elapsed time disclosed.

SOURCE / TRUTH CARD

One repository stays private until the video goes live. Verify signed-out source and model-download access before adding the description link. Original-code reuse terms still need a decision. Retain all upstream credits.

Before the take

Start the OpenJev vision service on localhost:8081 and the agency on localhost:8765. Reset with R. Select Four friends. Run Atlantic hideaway fresh. P pauses or resumes the presentation. Escape closes details. Warm both models before recording; cold load is separate from warm request timing.

The charts show saved benchmark results. The agency serves V1, while the final challenger result is V2. Photos, travellers and offers are fictional. Source-clause linkage is structured application logic.

Truth boundaries

Final references are synthetic and agent-reviewed, with 52 document repairs and two non-unanimous writer-assisted adjudications. All 360 scenarios remain. No human validation. The public workflow lane measures teacher agreement; upstream exposure may apply. Thirty-five workflow decisions failed the frozen context limit and remain in the denominator. Official TypeSafe benchmark scores were not reproduced.

Measured agency timing

Active V1: cold first HTTP request 10.1 seconds including loading. Three fresh forty-offer scans took 4.0, 4.2 and 4.5 seconds. Warm HTTP median 105 ms, 121 unique fresh requests, zero failures. These are text-only historical timings, separate from V2 model-only latency. The photo-aware 40-offer integration scan took 40.7 seconds in its recorded run.

Companion resource

Show the verified local bundle, including both checkpoints and original audit evidence. Add a public download link only once separately arranged. The OpenJev image branch ran locally; its pinned model is downloaded separately. The earlier text-only diffusion comparison was not executed. No public upload, deployment or messaging is implied.